

WEEK-2

Understanding Attention Mechanisms in Transformers

The Problem:

Traditional word embeddings assign a single vector to each word, but this fails to capture how meaning changes with context. Consider the word "bank" in these sentences:

Sentence 1: "I walked along the **bank** of the river."

Sentence 2: "I deposited money at the **bank** of America."

The same word has entirely different meanings riverside vs. financial institution yet traditional embeddings would give both instances the same representation.

Hence the , Self-Attention Mechanism

The Transformer's self-attention mechanism solves this by allowing each word to dynamically attend to all other words in the sentence, creating context-aware embeddings. Instead of static vectors, each word gets a unique representation that incorporates information from relevant surrounding words.

The Query-Key-Value Mechanism

For each word, the attention mechanism creates three vectors:

- **Query (Q):** What am I looking for? (represents the current word's search criteria)
- **Key (K):** Labels/catalog tags (represents what each word can provide)
- **Value (V):** The actual content/information (the actual content to be mixed)

Attention mechanism

Step 1: Compute Attention Scores

The model calculates how relevant each word is to "bank" by taking the dot product of bank's Query vector with every word's Key vector: For example,

Word	Score with "bank"	Normalized Weight
"river"	8.2	0.65
"the"	2.1	0.15
"of"	1.8	0.12
"bank"	3.5	0.08

"river" receives the highest attention weight (0.65), indicating strong relevance to "bank" in this context.

Steps to normalise, (briefly)

Step 1: Scale by $\sqrt{d}$

we divide scores by the square root of the dimension of the Key vectors ($\sqrt{d_k}$). This is called **scaled dot-product attention**.

$$\text{Scaled Score} = (\mathbf{Q} \cdot \mathbf{K}) / \sqrt{d_k}$$

Step 2: Softmax Normalization

The softmax function converts scaled scores into a probability distribution. It ensures all values are positive and sum to exactly 1.0.

$$\text{Softmax}(x_i) = e^{x_i} / \sum e^{x_j}$$

Step 2: Create a new (Context-Aware) Representation

The model creates a new representation of "bank" by taking a weighted sum of all Value vectors:

$$\text{bank}_{\text{new}} = 0.65 \times V_{\text{river}} + 0.15 \times V_{\text{the}} + 0.12 \times V_{\text{of}} + \dots$$

This weighted combination heavily incorporates the "river" context (65%), producing an embedding that represents "bank" as riverbank rather than a financial institution.

"Bank of America" , On the other hand,

In the sentence "deposited money at the bank of America," the attention weights shift dramatically:

For example,

- "deposited": 42%
- "money": 38%
- "America": 12%

The model now attends primarily to "deposited" (42%) and "money" (38%), producing an embedding aligned with financial institutions instead of geography.

Over all,

Self attention is dynamic and not Static which unlike traditional embeddings, it creates representations that change based on surrounding context

It also incorporates , parallel processing in which every word simultaneously computes attention with every other word, making Transformers highly parallelizable (unlike sequential RNNs).